

```

import numpy as np
import random
import matplotlib.pyplot as plt

# -----
# CONFIGURATION
# -----
GRID_SIZE = 20
NUM_COMPONENTS = 30
MAX_ITERS = 500
NEIGHBORHOOD = [(-1,0),(1,0),(0,-1),(0,1)] # neighbor moves

# -----
# NETLIST
# -----
def generate_netlist(num_components, connections_per_component=3):
    netlist = {}
    for i in range(num_components):
        possible = [j for j in range(num_components) if j != i]
        connected = random.sample(possible, min(connections_per_component, len(possible)))
        netlist[i] = connected
    return netlist

# -----
# FITNESS
# -----
def compute_wirelength(positions, netlist):
    total = 0
    for a, conns in netlist.items():
        for b in conns:
            total += np.sum(np.abs(np.array(positions[a]) - np.array(positions[b])))
    return total / len(netlist)

# -----
# PARALLEL CELLULAR ALGORITHM
# -----
def parallel_cellular_placement():
    netlist = generate_netlist(NUM_COMPONENTS)
    # Random initial placement
    positions = {i: (random.randint(0, GRID_SIZE-1), random.randint(0, GRID_SIZE-1))
                  for i in range(NUM_COMPONENTS)}

    history = []
    for iteration in range(MAX_ITERS):
        new_positions = positions.copy()
        changed = False # Track if any component moved

        for comp_id in range(NUM_COMPONENTS):

```

```

x, y = positions[comp_id]
best_pos = (x, y)
best_fit = compute_wirelength(positions, netlist)

for dx, dy in NEIGHBORHOOD:
    nx, ny = x + dx, y + dy
    if 0 <= nx < GRID_SIZE and 0 <= ny < GRID_SIZE:
        test_positions = positions.copy()
        test_positions[comp_id] = (nx, ny)
        test_fit = compute_wirelength(test_positions, netlist)
        if test_fit < best_fit:
            best_fit = test_fit
            best_pos = (nx, ny)
            changed = True

new_positions[comp_id] = best_pos

positions = new_positions
current_fit = compute_wirelength(positions, netlist)
history.append(current_fit)

if iteration % 50 == 0 or not changed:
    print(f"Iter {iteration:03d} | Fitness={current_fit:.3f}")

# Stop if no component moved
if not changed:
    print(f"☑ Converged after {iteration} iterations.")
    break

return positions, netlist, history

# -----
# RUN
# -----
best_positions, netlist, history = parallel_cellular_placement()

# -----
# VISUALIZE
# -----
x, y = zip(*best_positions.values())
plt.figure(figsize=(6,6))
plt.scatter(x, y, color="royalblue", s=50)
plt.title("Optimized Component Placement (Parallel Cellular Algorithm)")
plt.xlim(0, GRID_SIZE)
plt.ylim(0, GRID_SIZE)
plt.xlabel("X position")

plt.ylabel("Y position")

```

```
plt.grid(True)
```

```
plt.show()
```

```
plt.figure(figsize=(6,3))
```

```
plt.plot(history, color="darkorange")
```

```
plt.title("Fitness Evolution Over Iterations")
```

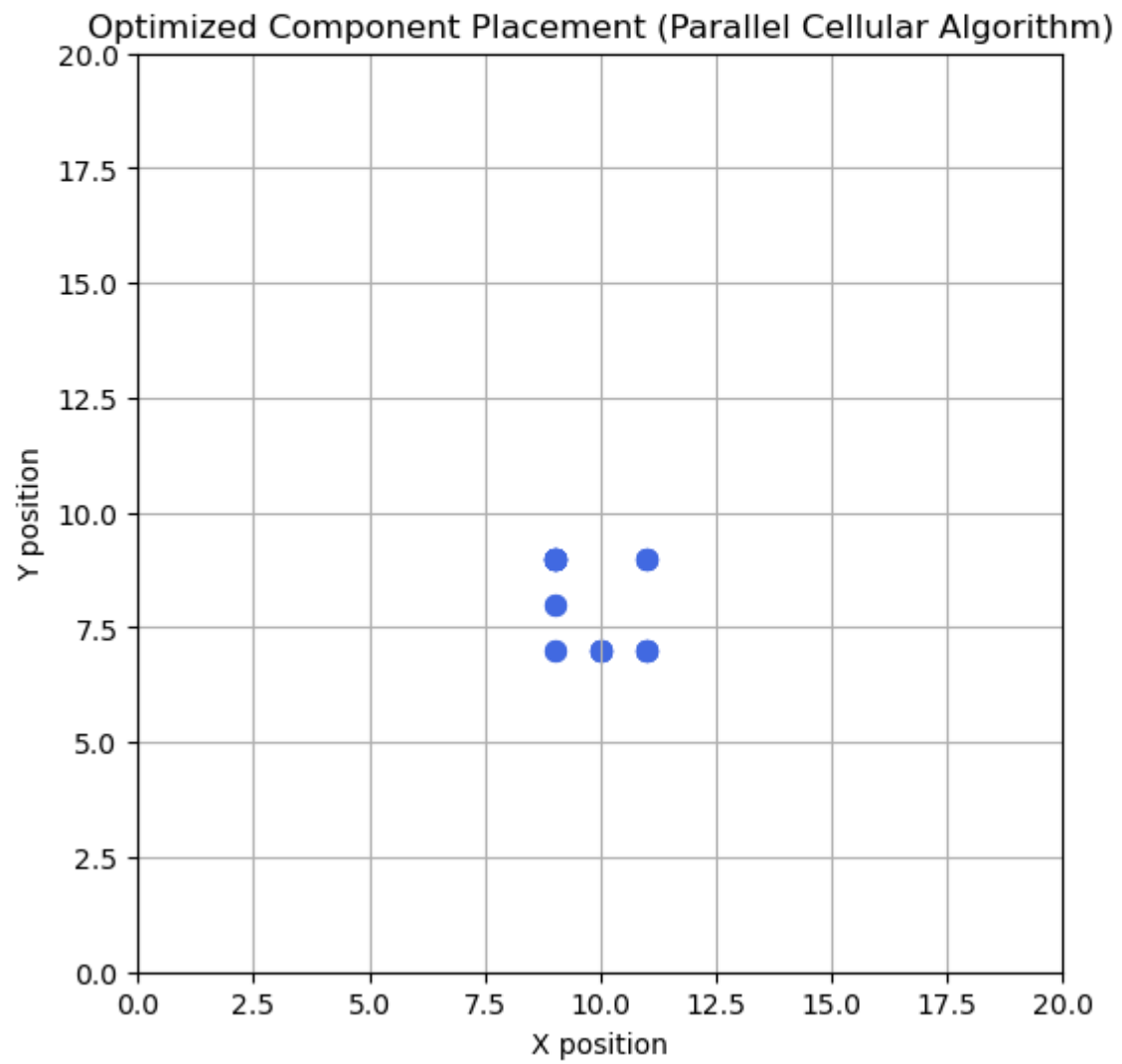
```
plt.xlabel("Iteration")
```

```
plt.ylabel("Wirelength (lower = better)")
```

```
plt.grid(True)
```

```
plt.show()
```

```
Iter 000 | Fitness=35.367
Iter 050 | Fitness=3.433
Iter 100 | Fitness=3.433
Iter 150 | Fitness=3.433
Iter 200 | Fitness=3.433
Iter 250 | Fitness=3.433
Iter 300 | Fitness=3.433
Iter 350 | Fitness=3.433
Iter 400 | Fitness=3.433
Iter 450 | Fitness=3.433
```



Fitness Evolution Over Iterations

